

1. Указатели и псевдоними в C++. Умни указатели. Масиви. Динамични масиви. Входно-изходни параметри на функции и методи.

1. Указатели в C++

1.1 Дефиния и деклариране

Указателят е променлива, която съхранява **адреса** на друга променлива в паметта. Всеки указател е асоцииран с определен тип данни - типът определя как да се интерпретира паметта, към която сочи.

```
int x = 22;  
int* ptr = &x;
```

Синтаксис за деклариране:

```
тип* идентификатор; // препоръчителен стил  
тип *идентификатор; // C-стил
```

1.2 Основни операции

Оператор	Значение
&	Адресен оператор - взема адреса на променливата
*	Оператор за разадресиране - достъп до стойността на адрес

=

Присвояване на адрес от един
указател на друг

```
int a = 10;
int* p = &a; // p = адреса на a
cout << p; // извежда адреса на (шестнайсетичен)
cout << *p; // извежда стойността на този адрес ( a = 10 )
*p = 20; // a е вече 20
cout << a; // 20
```

1.3 Нулев указател (nullptr)

Нулевият указател не сочи към никаква валидна памет в heap или stack:

```
int* p = nullptr; // C++ 11 и по-нови
int* p2 = NULL; // C-стил
```

Преди разадресиране винаги трябва да се проверява дали указателят е нулев:

```
if (p != nullptr) {
    cout << *p;
}
```

1.4 Аритметика с указатели

При аритметиката с указатели се придвижва с брой байтове, равен на размера на типа:

```
int arr[] = {10, 20, 30};
int* p = arr; // p сочи към arr[0]
p++; // p сочи към arr[1]
cout << *p; // 20
cout << *(p + 1); // 30
```

1.5 Указател към указател

```
int x = 5;
int* p = &x; // указател към int
int** pp = &p; // указател към указател към int

cout << x; // 5
cout << *p; // 5
cout << **pp; // 5

**pp = 99;
cout << x; // 99
```

Указателите към указатели се използват при динамични многомерни масиви и при функции, които трябва да модифицират самия указател (а не стойността, към която сочи).

2. Псевдоними (References) в C++

2.1 Дефинция

Псевдонимът е алтернативно име за вече съществуваща променлива. Работи се с него точно като с оригинала - без нужда от разадресиране.

Псевдонимите не съществуват в C, само в C++.

```
int a = 10;
int& ref = a; // ref е псевдоним към a

ref = 20;
cout << a; // 20
```

Ключови ограничения:

- Задължителна инициализация - не може да съществува непривързан псевдоним

- Не може да се пренасочва - веднъж привързан, псевдонимът остава свързан с оригиналната променлива
- Не може да се нулев

2.2 Константни псевдоними

```
int x = 5;  
const int& ref = x; // не може да се променя  
  
ref = 10; // грешка при компилация
```

Константните псевдоними могат да се инициализират с временни стойности (rvalue литерали). Неконстантните не могат.

```
const int& r = 42;    // валидно - животът на временния обект  
                    // се удължава  
// int& r2 = 42;      // ГРЕШКА - не може да е неконстантен
```

3. Умни указатели

Ръчното управление на паметта с `new` / `delete` е честа причина за изтичане на памет (memory leak), двойно освобождаване (double free) и dangling pointers. Умните указатели от `<memory>` решават тези проблеми - ресурсът се освобождава автоматично при унищожаване на обекта (изход от scope, хвърляне на изключение и т.н.).

3.1 `unique_ptr` - единствена собственост

- Само един `unique_ptr` може да притежава даден ресурс
- При унищожаване автоматично се извиква `delete`
- Не може да се копира, само да се премества с `std::move`
- Нулев `unique_ptr` = `nullptr`

```
#include <memory>

auto p1 = make_unique<int>(42);
cout << *p1; // 42

auto p2 = std::move(p1); // p1 е nullptr, а p2 притежава ресурсите
cout << (p1 == nullptr); // 1 (true)
cout << *p2; // 42

// Delete при излизане от scope
```

3.2 `shared_ptr` - споделена собственост

- Множество `shared_ptr` могат да сочат към един и същи обект
- Поддържа брояч на референции (reference count)
- Ресурсът се освобождава, когато броячът стигне 0

```
auto sp1 = make_shared<string>("hello"); // брояч = 1
auto sp2 = sp1; // брояч = 2
auto sp3 = sp1; // брояч = 3

cout << sp1.use_count(); // 3
sp2.reset(); // брояч = 2
sp3.reset(); // брояч = 1

// sp1 все още е валиден
```

3.3 `weak_ptr` - слаба референция

- Не участва в брояча на референции
- Основната употреба е за разбиване на циклични зависимости
- Трябва да се конвертира в `shared_ptr` чрез `lock()` преди употреба

```

auto sp = make_shared<int>(5);
weak_ptr<int> wp = sp;

cout << sp.use_count(); // 1 (тъй като wp не участва)

if (auto locked = wp.lock()) {    // проверка дали обектът е жив
    cout << *locked;    // 5
}

sp.reset();    // обектът е унищожен
auto locked2 = wp.lock();
cout << (locked2 == nullptr);    // 1 – обектът вече не съществува

```

3.4 Кой кога да се ползва?

- Unique Pointer - Единствена стойност; фабрични функции, локален scope
- Shared Pointer - Споделен ресурс между множество обекти
- Weak Pointer - Наблюдател без собственост; Служи за разбиване на циклични зависимости

4. Масиви

4.1 Едномерни масиви

Масивът е поредица от елементи с **еднакъв тип**, наредени последователно в паметта. Размерът трябва да е известен по **време на компилация**.

```

int arr[5] = {1, 2, 3, 4, 5}; // явна инициализация
int arr2[5] = {}; // всички елементи 0
int arr3[] = {1, 2, 3}; // размерът автоматично се извлича (3)

```

```
arr[0] = 100;
cout << arr[2]; // 3

int size = sizeof(arr) / sizeof(arr[0]);
```

Индексите са от 0 до N-1.

Връзка между масиви и указатели

Името на масива се "разпада" до указател към първи елемент:

```
int arr[] = {1, 2, 3};
int* p = arr;

cout << *p; // 1
cout << *(p + 1); // 2
cout << *(p + 2); // 3
```

4.2 Многомерни масиви

Многомерният масив е масив от масиви:

```
int matrix[3][4]; // 3 реда и 4 колони
int m[2][3] = {{1, 2, 3}, {4, 5, 6}}; // инициализация с вложени списъци
```

В паметта елементите са наредени последователно по редове:

```
m[0][0], m[0][1], m[0][2], m[1][0], m[1][1], m[1][2]
```

5. Динамични масиви

Статичните масиви изискват размерът да е известен по време на компилация. Когато размерът е известен едва по време на изпълнение (runtime), се използват динамични масиви, заделени в heap паметта.

5.1 Заделяне и освобождаване

```
int n;  
cin >> n;  
  
int* arr = new int[n]; // заделя n * sizeof(int) байта  
  
// Задължително освобождаване на паметта  
delete[] arr;  
  
// Мярка срещу видящи указатели  
arr = nullptr;
```

Критично правило: `new[]` → `delete[]` | `new` → `delete`. Смесването или пропускането води до изтичане на памет (memory leak) или неопределено поведение.

5.2 Динамичен многомерен масив

```
int rows = 3, cols = 4;  
int** matrix = new int*[rows];  
  
for (int i = 0 ; i < rows ; i++) {  
    matrix[i] = new int[cols];  
}  
  
for (int i = 0 ; i < rows ; i++) {  
    delete[] matrix[i];  
}  
  
delete[] matrix;  
matrix = nullptr;
```

6. Входно-изходни параметри на функции и методи

При извикване на функция параметрите могат да се предадат по три начина, всеки с различни характеристики по отношение на ефективност, безопасност и семантика:

6.1 Предаване по стойност (Pass by value)

Функцията получава копие на аргумента. Промените в копието не се отразяват на оригинала.

```
void increment(int x) {  
    x++; // променя само локалното копие  
}  
  
int a = 5;  
  
increment(a);  
  
cout << a; // a = 5;
```

Недостатък: При големи структури (`std::vector` или `std::string`) копирането е скъпо.

6.2 Предаване по указател (Pass by Pointer)

Функцията получава адреса на аргумента. Чрез разадресиране може да се модифицира оригинала.

```
void increment(int* x) {  
    if (x != nullptr) {  
        (*x)++; // модифицира оригинала  
    }  
}
```

```
int a = 5;
increment(&a); // предаваме адрес
cout << a; // 6
```

Подходящо за: Когато параметърът може да е `nullptr` (незадължителен); при работа с масиви; при нужда от пренасочване на самия указател.

6.3 Предаване по псевдоним (Pass by Reference)

Функцията работи директно с оригиналната променлива. Синтактично по-чисто от указатели.

```
void increment(int& x) {
    x++;
}

int a = 5;
increment(a); //
cout << a; // a = 6
```

Подходящо за: Когато параметърът задължително трябва да е валиден; входно-изходни параметри; замяна на указател при по-прост синтаксис.

6.4 Предаване по константен псевдоним (Pass by Constant Reference)

Позволява четене без копиране - ефективно за всякакви обекти:

```
void print(const string& s) {
    cout << s;
    // не може да правим s = "нова стойност"
}

string name = "Александър"
print(name); // без копиране на целия string
```

Подходящо за: Входни параметри на по-сложни типове - `std::string`, `std::vector`, класове.

6.5 Входни / Изходни / вход-изходни параметри

- Входен
 - `int x` или `const int& x`
 - Само четене
- Изходен
 - `int& result` или `int* result`
 - Само запис
- Вход-изход
 - `int& x` или `int* x`
 - Четене и запис

```
void mySwap(int& a, int& b) {  
    int temp = a;  
    a = b;  
    b = temp;  
}  
  
int x = 3, y = 7;  
mySwap(x, y);  
  
bool safeDivide(double a, double b, double& result) {  
    result = a / b;  
    return true;  
}  
  
double res;  
  
safeDivide(10.0, 3.0, res);
```

6.6 Предаване на масиви към функции

При подаването на масив, той се "разпада" до указател - информацията се губи. Затова размерът се подава отделно.

```
void fillArray(int* arr, int size, int value) {  
    //  
}  
  
int arr[5];  
fillArray(arr, 5, 0);
```